

JavaScript

Import & Export Modules

Simple Guide for Beginners

Short words. Clear examples. Easy to follow.

1. What is a Module?

A **module** is just a JavaScript file that holds some code.

When your project gets big, it is hard to keep all your code in one file. So you split it into small files. Each small file is called a **module**.

Real-life example: Think of a restaurant menu. Instead of one huge list, there are separate pages — Starters, Main Course, Desserts. Each page is like a module: it holds only its own items.

Why use modules?

Reason	What it means
Reuse code	Write a function once. Use it in many files.
Easy to fix	Small files are easier to read and fix.
No name clash	Each file keeps its own variables. No mix-up.
Stay organised	Each file does one job. Everything has a place.

2. export — Share Your Code

The word **export** tells JavaScript: *"This piece of code can be used by other files."*

Without export, the code stays private inside that file only.

Type 1 — Named Export

You put the word **export** in front of anything you want to share. You can share many things from one file.

```
// math.js

export const PI = 3.14;           // share a number

export function add(a, b) {       // share a function
  return a + b;
}

export function multiply(a, b) { // share another function
  return a * b;
}
```

You can also write all exports together at the bottom:

```
// math.js (another way to write it)

const PI = 3.14;
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }

export { PI, add, multiply }; // share all at once
```

Type 2 — Default Export

Each file can have **one** special main export. We call it the **default** export. Use this for the most important thing in the file.

```
// greet.js

export default function greet(name) {
  return 'Hello, ' + name + '!';
}
```

Remember: One file can have MANY named exports, but only ONE default export.

3. import — Use Code From Another File

The word **import** brings code from another file into your current file. It is like borrowing a tool from a neighbour.

Way 1 — Import Named Exports

Write the names inside `{ }` curly brackets. The names must match exactly.

```
// app.js

import { PI, add, multiply } from './math.js';
```

```
console.log(PI);           // shows: 3.14
console.log(add(2, 3));    // shows: 5
console.log(multiply(4, 5)); // shows: 20
```

Way 2 — Import with a New Name (alias)

Sometimes two files have the same name. Use **as** to rename it.

```
import { add as sum, multiply as times } from './math.js';

console.log(sum(10, 5)); // shows: 15
console.log(times(3, 4)); // shows: 12
```

Way 3 — Import Default Export

No curly brackets needed. You can choose any name you like.

```
import greet from './greet.js';

console.log(greet('Alice')); // shows: Hello, Alice!
console.log(greet('Bob'));   // shows: Hello, Bob!
```

Way 4 — Import Everything at Once

Use *** as name** to get everything from a file in one go.

```
import * as Math from './math.js';

console.log(Math.PI);           // shows: 3.14
console.log(Math.add(5, 5));    // shows: 10
console.log(Math.multiply(3, 3)); // shows: 9
```

4. Full Example — 3 Files Working Together

Here is a small project with 3 files. Read top to bottom and follow the flow.

File 1: utils.js (helper tools)

```
// utils.js

// Make any text ALL CAPS
export function makeUpper(text) {
  return text.toUpperCase();
}

// Reverse any text (hello -> olleh)
export function reverseText(text) {
  return text.split('').reverse().join('');
}

// App version number
export const VERSION = '1.0.0';
```

File 2: user.js (User class — default export)

```
// user.js

import { makeUpper } from './utils.js'; // use a helper

export default class User {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    // makeUpper turns the name to capitals
    return 'Hi! I am ' + makeUpper(this.name) + ', age ' + this.age;
  }
}
```

File 3: main.js (start here)

```
// main.js

import User from './user.js'; // default import
import { reverseText, VERSION } from './utils.js'; // named imports

const u = new User('alice', 25);
console.log(u.greet());
// shows: Hi! I am ALICE, age 25

console.log(reverseText('hello')); // shows: olleh
console.log('Version:', VERSION); // shows: Version: 1.0.0
```

5. Named vs Default — Side by Side

Question	Named Export	Default Export
How to export?	<code>export { foo }</code>	<code>export default foo</code>
How to import?	<code>import { foo } from '...'</code>	<code>import anything from '...'</code>
How many per file?	As many as you need	Only ONE
Can I rename it?	<code>Yes: import { foo as f }</code>	Yes — any name you like
Best for?	Helper functions, constants	The main class or function

6. Dynamic Import — Load Only When Needed

Normal imports load at the start, every time. **Dynamic import** loads a file *only when you actually need it*. This makes your app faster.

```
// Load math.js only when the user clicks a button

async function loadMath() {
  const mathModule = await import('./math.js');
  console.log(mathModule.add(10, 20)); // shows: 30
}

document.getElementById('btn').addEventListener('click', loadMath);
```

Tip: Dynamic import always returns a Promise. So use `async / await`, or `.then()` to get the result.

7. How to Use Modules in a Web Page

To use modules in an HTML page, just add **type="module"** inside your script tag. That is the only change you need.

```
<!-- index.html -->

<script type="module" src="main.js"></script>
```

Good to know: When you use `type="module"`, the script runs AFTER the whole page loads. So you do not need to worry about the order of things.

8. Common Mistakes (and How to Fix Them)

Mistake	Wrong code	Correct code
Using {} for default import	<pre>import { greet } from './greet.js'</pre>	<pre>import greet from './greet.js'</pre>
No {} for named import	<pre>import add from './math.js'</pre>	<pre>import { add } from './math.js'</pre>
Missing .js at the end	<pre>import { add } from './math'</pre>	<pre>import { add } from './math.js'</pre>
Two default exports	<pre>export default a; export default b;</pre>	Only ONE default export per file!

9. Quick Cheat Sheet — All Syntax at a Glance

[illegible]

[illegible]

You did it! Now you know how to share and use code across JavaScript files.